Tutorials

The tutorials in this section describe how to write remote-running robot applications using Misty's HTTP API and WebSocket connections. While Misty can receive HTTP API commands from any device on her local network, in these tutorials we use .html files and in-line JavaScript to build applications that you can run from any web browser.

Changing Misty's LED

In this tutorial, you learn how to write a program that sends a REST command to change the color of Misty's chest LED.

Connecting Misty to Your Network

Because these commands are sent to Misty over a local network connection, you must connect your robot to your local network. Use Misty Connect to connect your robot to your Wi-Fi network, or follow this guide to connect Misty to your Wi-Fi network using the Command Center and an Ethernet/USB dongle. Once Misty is connected to your network, take note of her IP address to use with the HTTP API commands.

Setting Up Your Project

This tutorial uses Misty's HTTP API to send a POST request that changes the color of her chest LED and logs a successful response. To set up your project, create a new .html document. To simplify the task of making `XMLHttpRequests` calls to Misty from the browser, we use Axios, an HTTP library supported by most web browsers and Node.js. To use Axios in your program, reference a link to a content delivery network (CDN) for Axios inside `<script>` tags in the `<head>` section of your .html file when you set up the project.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Tutorial | Changing Misty's LED</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Reference a link to a CDN for Axios here -->
    <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
</head>
<body>
</body>
</html>
```

HTML

Alternately, you can download a compressed version of the Axios library to include in your project. Read more about Axios here.

Writing the Code

Within `<script>` tags in the `<body>` of your .html document, declare a constant variable `ip` and set its value to a string with your robot's IP address. We'll reference this variable throughout the program to send commands to Misty.

```
// Declare a constant variable.
// Set its value to your robot's IP address.
const ip = "<robotipaddress>";
```

JavaScript

When we send a command to change Misty's LED color, we need to communicate what the new color should be. The HTTP API command to change Misty's LED requires three parameters: `"red"`, `"green"`, and `"blue"`. These parameters represent the RGB values of the new color.

Create an object called `data` to send with the POST request. Create a property for each color parameter, and set the value of each property to an integer between `0` and `255`. The RGB values in the example change Misty's chest LED to hot pink.

```
// Create a data object to send with the POST request.
// Set values for each RGB color property.
let data = {
    "red": 255,
    "green": 0,
    "blue": 255
};
```

JavaScript

Now we're ready to write the code to send the command to Misty. We do this by using the `axios.post()` method included in the Axios library. This method accepts two parameters:

- the URL of the request, and
- the data to send with the request.

The HTTP API endpoint for the `ChangeLED` command is `http://<robotipaddress>/api/led`. In your code, call `axios.post()` and pass a string with this endpoint as the first parameter. Use the previously defined variable `ip` to populate the `<robotipaddress>` section of the URL. Pass the `data` object for the second parameter.

```
// Call axios.post(). Pass the URL of the ChangeLED
// endpoint as the first parameter and the data object
// as the second.
axios.post("http://" + ip + "/api/led", data)
```

JavaScript

Because Axios is promise based, we need to use a `then()` method after calling `axios.post()`. This method returns a promise and triggers a callback function if the promise is fulfilled. We pass a callback function to `then()` to interpret information from the return values of the POST call and print a message to the console about whether the request was a failure or success.

```
// Use a then() method after calling axios.post().
// Pass in a callback function to interpret the return
// values of the call and to print a message to the console
// indicating the request's success.
axios.post("http://" + ip + "/api/led", data)
    .then(function (response) {
        console.log(`ChangeLED was a ${response.data.status}`);
    })
```

JavaScript

We use a `catch()` method after `then()`, which triggers if the promise is rejected. Pass a callback function to `catch()` to print to the console any errors returned by the request.

```
// Use a catch() method after then(). catch() triggers
// if the promise is rejected. Pass a callback to catch()
// to print any errors to the console.
axios.post("http://" + ip + "/api/led", data)
    .then(function (response) {
        console.log(`ChangeLED was a ${response.data.status}`);
    })
    .catch(function (error) {
```

```
        console.log(`There was an error with the request ${error}`);
    })
```

JavaScript

Now we're ready to run the program!

1. Save your .html document.
2. Open the .html file in a web browser.
3. Open the developer tools of your web browser to view the console.

When the page loads, it sends a `ChangeLED` command to Misty, and a message about the results of the command prints to the console. **Congratulations!** You have just written your first program using Misty's HTTP API!

Full Sample

Download the full .html document from GitHub.

Using Sensors, WebSockets, and Locomotion

In this tutorial, we write a skill that commands Misty to drive in a straight line for a designated period of time and stop if she encounters an object in her path. We do this by combining Misty's `DriveTime` locomotion command with information received from the `TimeOfFlight` and `LocomotionCommand` WebSocket connections. In this tutorial, you'll learn:

- How to subscribe to data from Misty's WebSocket connections
- How to use the `lightSocket.js` helper tool
- How to write callbacks that use data from WebSocket connections to allow Misty to make decisions about what to do in different situations

Before you write any code, connect Misty to your home network and make sure you know her IP address. You can see how to get this information in the first tutorial above.

Setting Up Your Project

In addition to Axios, this project uses the `lightSocket.js` helper tool to simplify the process of subscribing to Misty's WebSocket streams. You can download this tool from our GitHub repository. Save the `lightSocket.js` file to a “tools” or “assets” folder in your project.

To set up your project, create a new .html document. Give it a title, and include references to `lightSocket.js` and a content delivery network (CDN) for the Axios library in the `<head>` section. We write the code for commanding Misty within `<script>` tags in the `<body>` section of this document.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Tutorial | Using Sensors, WebSockets, and
Locomotion</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Include references to a CDN for the Axios library and the local
path where lightSocket.js is saved in the <head> of your document -->
    <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
    <script src="<local-path-to-lightSocket.js"></script>
    </head>
<body>
    <script>
    // The code for commanding Misty goes here!
    </script>
</body>
```

HTML

Writing the Code

Within `<script>` tags in the `<body>` of your document, declare a constant variable `ip` and set its value to a string with your robot's IP address. We use this variable to send commands to Misty.

```
// Declare a constant variable and set its
// value to a string with your robot's IP address.
const ip = "<robotipaddress>";
```

JavaScript

OPENING A CONNECTION

Create a new instance of `LightSocket` called `socket`. The `socket` instance takes as parameters the IP address of your robot and two optional callback functions. The first callback triggers when a connection is opened, and the second triggers when it's closed. Pass `ip` and a function called `openCallback()` to the new instance of `LightSocket`. Below these declarations, declare the `openCallback()` function.

```
// Create a new instance of LightSocket called
// socket. Pass as arguments the ip variable and
// a function named openCallback.
let socket = new LightSocket(ip, openCallback);

/* COMMANDS */

// Define the function passed as the callback
// to the new instance of LightSocket. This is
// the code that executes when socket opens a
// connection to your robot.
function openCallback() {

}
```

JavaScript

Once a connection is opened, we want to do three things:

- Subscribe to the `TimeOfFlight` WebSocket.
- Subscribe to the `LocomotionCommand` WebSocket.
- Send Misty a `DriveTime` command.

We write the code for this inside the `openCallback()` function.

SUBSCRIBING TO WEBSOCKETS

Let's start by subscribing to Misty's `TimeOfFlight` and `LocomotionCommand` WebSocket connections.

The `TimeOfFlight` WebSocket sends data from the time-of-flight (TOF) sensors around Misty's base. These sensors tell Misty how far objects are away from her, or if she's about to drive off a ledge. For this program, we're interested in receiving data from Misty's front center TOF sensor. This sensor points straight forward in Misty's direction of travel.

The instance of `LightSocket` we've created (called `socket`) uses the `Subscribe()` method to subscribe to WebSocket connections. The `Subscribe()` method takes 8 parameters.

```
socket.Subscribe(eventName, msgType, debounceMs, property,
inequality, value, [returnProperty], [eventCallback])
```

JavaScript

Note that these parameters correlate with the values required in the subscribe message you send when you open a WebSocket without LightSocket, described in the documentation here. `LightSocket` simplifies the process by taking your parameters and generating a properly formatted subscribe message.

To subscribe to the data stream from `TimeOfFlight`, call the `Subscribe()` method on `socket`. Pass the following for each parameter:

1. `eventName` is a string that designates the name you would like to give this event. Choose a unique name that indicates the function the event serves. Let's call our event `"CenterTimeOfFlight"`.
2. `msgType` is a string that specifies the WebSocket data stream to subscribe to. We're subscribing to Misty's `"TimeOfFlight"` WebSocket.
3. `debounceMs` specifies how often in milliseconds Misty should send a message with `TimeOfFlight` data. Enter `100` to receive a message every tenth of a second. At the speed we command Misty to travel, this should be precise enough for us to be able to execute a `Stop` command before Misty collides with an object in her path.
4. The fourth, fifth, and sixth parameters form a comparison statement that specifies event conditions to filter out unwanted messages. The `TimeOfFlight` WebSocket data stream can send data from all of Misty's TOF sensors, but we only need data from her front center sensor. Pass `"SensorPosition"` for the `property` parameter to specify we want data from a specific sensor.
5. `inequality` is a string that sets a comparison operator to specify the conditions of events to receive messages about. In this case we use `"=="`.
6. `value` is a string that specifies which value of the `property` parameter to check against. We want to receive information for TOF sensors where the value of the `"SensorPosition"` property is `"Center"`.
7. `returnProperty` is an optional parameter. We don't need to pass an argument for this parameter for our subscription to `TimeOfFlight`. Enter `null`.
8. The parameter `eventCallback` is for the callback function that triggers when WebSocket data is received. Name this function `_centerTimeOfFlight()` to correspond to the name we provided for this event. The **Callbacks** section of this tutorial describes how to write the code for this function.

```
function openCallback() {

    // Print a message when the connection is opened.
    console.log("socket opened");

    // Subscribe to an event called CenterTimeOfFlight
    // that returns TimeOfFlight data. Pass arguments
    // to make sure this event returns data for the
    // front center time-of-flight sensor every 100
    // milliseconds. Pass the callback function
    // _centerTimeOfFlight() as the final argument.
    socket.Subscribe("CenterTimeOfFlight", "TimeOfFlight", 100,
"SensorPosition", "==", "Center", null, _centerTimeOfFlight);
}
```

JavaScript

The `LocomotionCommand` WebSocket sends data every time the robot's linear or angular velocity changes (see the documentation here for more information). We use this WebSocket to learn when Misty has stopped moving.

As with `TimeOfFlight`, we need to pass eight parameters to `socket.Subscribe()` to receive data from `LocomotionCommand`. However, because we only want to know whether Misty's movement has changed, we don't need to filter our results to specific event properties. We only need to pass arguments for `eventName` (`"LocomotionCommand"`), the WebSocket name (also `"LocomotionCommand"`), and the `eventCallback` function, which we call `_locomotionCommand()`. Enter `null` for all of the other parameters.

```
function openCallback() {

    socket.Subscribe("CenterTimeOfFlight", "TimeOfFlight", 100,
"SensorPosition", "==", "Center", null, _centerTimeOfFlight);

    // Subscribe to an event called LocomotionCommand
    // that returns data when Misty's angular or linear
    // velocity changes. Pass the callback function
    // _locomotionCommand() as the final argument.
    socket.Subscribe("LocomotionCommand", "LocomotionCommand", null, null,
null, null, null, _locomotionCommand);

}
```

JavaScript

SENDING COMMANDS

After we've subscribed to these WebSockets, we issue the command for Misty to drive by using Axios to send a POST request to the `DriveTime` endpoint. This endpoint accepts values for three properties: `LinearVelocity`, `AngularVelocity`, and `TimeMS`. Inside the `openCallback()` function, create a `data` object with the following key/value pairs to send with the REST command:

- Set `LinearVelocity` to `50` to tell Misty to drive forward at a moderate speed.
- Set `AngularVelocity` to `0`, so Misty drives straight without turning.
- Set `TimeMS` to `5000` to specify that Misty should drive for five seconds.

```
function openCallback() {

    socket.Subscribe("CenterTimeOfFlight", "TimeOfFlight", 100,
"SensorPosition", "==", "Center", null, _centerTimeOfFlight);

    socket.Subscribe("LocomotionCommand", "LocomotionCommand", null, null,
null, null, null, _locomotionCommand);

    // Create a data object to send with the DriveTime command.
    let data = {
        LinearVelocity: 50,
        AngularVelocity: 0,
        TimeMS: 5000
    };

}
```

JavaScript

> **Note:** You can learn more about `DriveTime` and how the parameters affect Misty's movement in the API section of this documentation.

Pass the URL for the `DriveTime` command along with this `data` object to the `axios.post()` method. Use a `then()` method to handle a successful response and `catch()` to handle any errors.

```
function openCallback() {

    socket.Subscribe("CenterTimeOfFlight", "TimeOfFlight", 100,
"SensorPosition", "==", "Center", null, _centerTimeOfFlight);

    socket.Subscribe("LocomotionCommand", "LocomotionCommand", null, null,
null, null, null, _locomotionCommand);

    let data = {
```

```
            LinearVelocity: 50,
            AngularVelocity: 0,
            TimeMS: 5000
        };

        // Use axios.post() to send the data
        // to the endpoint for the DriveTime command.
        axios.post("http://" + ip + "/api/drive/time", data)
            // Use .then() to handle a successful response.
            .then(function (response) {
                // Print the results
                console.log(`DriveTime was a ${response.data.status}`);
            })
            // Use .catch() to handle errors
            .catch(function (error) {
                // Print any errors
                console.log(`There was an error with the request ${error}`);
            });
};
```

JavaScript

SETTING UP CALLBACKS

Now that we've written the code to subscribe to the WebSocket connections and send the `DriveTime` command, we're ready to write the callback functions `_centerTimeOfFlight()` and `_locomotionCommand()`. These functions trigger when Misty sends data for the events we've subscribed to.

Start with `_centerTimeOfFlight()`, the callback function passed to `socket.Subscribe()` for the `TimeOfFlight` WebSocket connection. We subscribe to the `CenterTimeOfFlight` event in order to examine incoming data and tell Misty what to do when she detects an object in her path. Data from this WebSocket is passed directly into the `_centerTimeOfFlight()` callback function. `_centerTimeOfFlight()` should parse this data and send Misty a `Stop` command if an object is detected in her path.

We define our callbacks above the section where we define our commands. Create a function called `_centerTimeOfFlight()` with a single parameter called `data`. This parameter represents the data passed to Misty when the `CenterTimeOfFlight` event triggers.

```
/* CALLBACKS */

// Define the callback function for handling
// CenterTimeOfFlight event data
let _centerTimeOfFlight = function (data) {
};
```

JavaScript

When you subscribe to an event, some messages come through that don't contain event data. These are typically registration or error messages. To ignore these messages, place the code for our callback function in `try` and `catch` statements.

```
let _centerTimeOfFlight = function (data) {
    // Use try and catch statements to handle
    // exceptions and unimportant messages
    // from the WebSocket data stream.
    try {

    };
    catch(e) {

    };
};
```

JavaScript

Inside the `try` statement, instantiate a `distance` variable. `distance` stores a value representing the distance from Misty in meters an object has been detected by her front center time-of-flight sensor. This value is stored in the `data` response at `data.message.distanceInMeters`. Log `distance` to the console.

```
let _centerTimeOfFlight = function (data) {
    try {
        // Create a distance variable to store
        // the value representing the distance
        // from Misty in meters an object has been
        // detected by her front center time-of-flight sensor.
        let distance = data.message.distanceInMeters;
        // Print this distance to the console.
        console.log(distance);
    };
    catch(e) { };
};
```

JavaScript

We only want Misty to stop when `distance` is a very small value, indicating she is very close to an object. To do this, write an `if` statement to check if `distance < 0.2`. Misty keeps driving if `distance` is greater than `0.2` meters. If `distance` is `undefined`, an exception occurs and is passed to the `catch` statement. This is the case when registration or error messages are received through the WebSocket. By using a `try` statement, our callback functions behave appropriately when the “right” messages come through, and continue execution if they cannot act on the data they receive.

If `distance` is a value less than `0.2`, use Axios to issue a POST request to the endpoint for the `Stop` command: `"http://" + ip + "/api/drive/stop"`. This endpoint does not require parameters, so we can omit the second parameter of `axios.post()`. Use `then()` and `catch()` to log successful responses and catch potential errors.

```
let _centerTimeOfFlight = function (data) {
    try {
        let distance = data.message.distanceInMeters;
        console.log(distance);
        // Write an if statement to check
        // if the distance is smaller than 0.2 meters.
        if (distance < 0.2) {
            // If the instance is less than 0.2 meters, send
            // a request to endpoint for the Stop command.
            axios.post("http://" + ip + "/api/drive/stop")
                .then(function (response) {
                    // Print the results
                    console.log(`Stop was a ${response.data.status}`);
                })
                .catch(function (error) {
                    // Print any errors
                    console.log(`There was an error with the request
${error}`);
                });
            }
    };
    catch(e) { };
};
```

JavaScript

The `_centerTimeOfFlight()` callback triggers every time data from Misty's front center sensor is received. If an object is detected close enough to the sensor, a `Stop` command is issued, and Misty stops before colliding with the object.

The purpose of the `_locomotionCommand()` callback function is to “clean up” our skill when the program stops executing. Whenever you subscribe to a WebSocket, you should

unsubscribe when you are done with it, so Misty stops sending data. Our program can end in two ways:

- Misty stops driving when she detects an object in her path.
- Misty does not detect an object in her path and stops driving after five seconds.

The `LocomotionCommand` event sends data whenever linear or angular velocity changes, including when Misty starts moving when the program starts. We want to unsubscribe from WebSocket connections when Misty stops **and** the value of `LinearVelocity` is `0`. Declare a function called `_locomotionCommand()`, and pass it a parameter for the `data` received by the `LocomotionCommand` WebSocket. We only want to unsubscribe when Misty stops, so we add the condition that `linearVelocity` should be `0` to an `if` statement. As with the `_centerTimeOfFlight()` callback, place this condition inside a `try` statement, and place a `catch` statement to handle exceptions at the end of the function.

```
// Define the callback function passed in
// the subscription to LocomotionCommand events
let _locomotionCommand = function (data) {
    // Use try and catch statements to handle
    // exceptions and unimportant messages
    try {
        // Use an if statement to check
        // whether Misty stopped moving
        if (data.message.linearVelocity === 0) {

        }
    }
    catch(e) {

    }
};
```

JavaScript

If `data.message.linearVelocity === 0`, the program should unsubscribe from the WebSocket connections we've opened. Write commands to unsubscribe from the `DriveTime` and `LocomotionCommand` WebSocket connections, and log a message to the console so you can verify that this only happens when `linearVelocity` is indeed `0`.

```
let _locomotionCommand = function (data) {
    try {
        if (data.message.linearVelocity === 0) {
            // Print a message to the console to aid in debugging
            console.log("LocomotionCommand received linear velocity as",
data.message.linearVelocity);
            // Unsubscribe from the CenterTimeOfFlight
            // and LocomotionCommand events
            socket.Unsubscribe("CenterTimeOfFlight");
            socket.Unsubscribe("LocomotionCommand");
        }
    }
    catch(e) {

    }
};
```

JavaScript

At the bottom of the script, call `socket.Connect()`. When the connection is established, the `openCallback()` function executes to subscribe to WebSocket connections and send Misty a `DriveTime` command. Data received through WebSocket connections is passed to the `_centerTimeOfFlight()` and `_locomotionCommand()` callback functions.

```
// Open the connection to your robot.
// When the connection is established,
// the openCallback function executes
```

```
// to subscribe to WebSockets and send
// Misty a DriveTime command. Event data
// is passed to the _centerTimeOfFlight() and
// _locomotionCommand() callback functions.
socket.Connect();
```

JavaScript

Congratulations! You've just written another skill for Misty. Save your .html document and open it in a web browser to watch Misty go. When the document loads, the program:

- connects with Misty
- sends a `DriveTime` command for Misty to drive forward for 5 seconds
- subscribes to `TimeOfFlight` events to detect if an object is in Misty's path and sends a `Stop` command if so
- subscribes to `LocomotionCommand` to detect when Misty has come to a stop and unsubscribe from the WebSocket connections

Full Sample

Download the full .html document from GitHub.

Exploring Computer Vision

This tutorial teaches how to write a skill to have Misty detect, recognize, and learn faces. When this skill runs, Misty checks a given name against her list of known faces. If the name exists, she engages facial recognition to see the user in her field of vision and print a message to the console, greeting the user by name. If the name does not match a known face, Misty uses facial training to learn the user's face, assigns it the name provided, and prints a greeting to the console. This tutorial teaches

- how to use HTTP API commands for facial training and recognition
- how to subscribe to and use data from Misty's `FaceRecognition` WebSocket connection

Before you write any code, connect Misty to your home network and make sure you know her IP address. You can see how to get this information in the first tutorial above.

Setting Up Your Project

This project uses the Axios library and the `lightSocket.js` helper tool to handle requests and simplify the process of subscribing to Misty's WebSocket connections. You can read more about these tools in the first and second tutorials above.

To set up your project, create a new .html document. Give it a title, and include references to `lightSocket.js` and a content delivery network (CDN) for the Axios library in the `<head>` section. Place the code for commanding Misty within `<script>` tags in the `<body>` section of this document.

```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="utf-8" />
        <title>Tutorial | Exploring Computer Vision</title>
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <!-- Include references to a CDN for the Axios library and the local
path where lightSocket.js is saved in the <head> of your document -->
        <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
        <script src="<local-path-to-lightSocket.js>"></script>
    </head>
    <body>
        <script>
           // Write the code for the program here!
       </script>
```

```
    </body>
</html>
```

HTML

Writing the Code

Within `<script>` tags in the `<body>` of your document, declare a constant variable `ip` and set its value to a string with your robot's IP address. We use this variable to send commands to Misty.

```
/* GLOBALS */

// Declare a constant variable and set its
// value to a string with your robot's IP address.
const ip = "<robotipaddress>"
```

JavaScript

Create a global constant called `you` and assign it to a string with your name. Initialize an additional global variable called `onList` with the value `false`. We use these variables to check and indicate whether the user (`you`) is found on Misty's list of learned faces.

```
/* GLOBALS */

const ip = "<robotipaddress>"

// Create a global constant called you
// and assign it to a string with your name.
const you = "<your-name>"

// Initialize another variable called
// onList and set its value to false.
let onList = false;
```

JavaScript

> **Note:** Avoid hard-coding name values like this in real-world applications of Misty skills. Instead, create a form in the browser where users can type and send their names to Misty.

OPENING A CONNECTION

Beneath these global variable declarations, declare a new instance of `LightSocket` called `socket`. This instance of `LightSocket` takes as parameters your robot's IP address and callback functions that trigger when the connection opens or closes. Pass `ip` as the first argument, and specify a parameter for the open callback function named `openCallback()`. Below this declaration, declare the `openCallback()` function with the prefix `async` to indicate it is an asynchronous function.

```
// Create a new instance of LightSocket called
// socket. Pass as arguments the ip variable
// and a function named openCallback.
let socket = new LightSocket(ip, openCallback);

/* CALLBACKS */

// Define the function passed as the callback
// to the new instance of LightSocket. This is
// the code that executes when socket opens a
// connection to your robot.
async function openCallback() {
```

```
}
```

JavaScript

A subscription to the `FaceRecognition` WebSocket may already be active if the skill has run multiple times in quick succession, or if the program crashed before reaching completion. To handle this, pass `"FaceRecognition"` to `socket.Unsubscribe()` at the beginning of the `openCallback()` function. This unsubscribes from any existing `FaceRecognition` WebSocket connections to avoid issues caused by multiple attempts to subscribe to the same event.

```
async function openCallback() {
    // Unsubscribe from any existing FaceRecognition
    // WebSocket connections.
    socket.Unsubscribe("FaceRecognition");
}
```

JavaScript

Next, the program should pause to give Misty time to register and execute the command. Do this by defining a helper function called `sleep()`. The `sleep()` function creates and returns a promise that resolves when `setTimeout()` expires after a designated number of milliseconds. Declare this function at the top of your script so other parts of the program can access it. Inside the `openCallback()` function, call the `sleep()` function and pass in a value of `3000`. Prefix `sleep()` with `await` to indicate that `openCallback()` should pause execution of the event loop until the promise has been resolved.

```
/* TIMEOUT */

// Define a helper function called sleep that
// can pause code execution for a period of time.
function sleep(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

/* CALLBACKS */

async function openCallback() {
    socket.Unsubscribe("FaceRecognition");
    // Use sleep() to pause execution for
    // three seconds to give Misty time
    // to register and execute the command.
    await sleep(3000);
}
```

JavaScript

Next, check if the name stored in `you` is included on the list of faces Misty already knows. Inside `openCallback()`, use Axios to issue a GET request to the endpoint for the `GetKnownFaces` command: `"http://" + ip + "/api/faces"`.

```
async function openCallback() {
    socket.Unsubscribe("FaceRecognition");
    await sleep(3000);

    // Issue a GET request to the endpoint
    // for the GetKnownFaces command.
    axios.get("http://" + ip + "/api/faces")
}
```

JavaScript

This request returns a list of the names of faces Misty has already been trained to recognize. We pass a callback function to a `then()` method to parse the response to the `GetKnownFaces` request, and check whether the name stored in `you` exists in Misty's list of

known faces. Start by storing the list returned by the response in a variable called `faceArr`. Print `faceArr` to the console.

```
async function openCallback() {
    socket.Unsubscribe("FaceRecognition");
    await sleep(3000);

    // Use then() to pass the response
    // to a callback function.
    axios.get("http://" + ip + "/api/faces").then(function (res) {
        // Store the list of known faces in the
        // faceArr variable and print this list.
        let faceArr = res.data.result;
        console.log("Learned faces:", faceArr);
    });
}
```

JavaScript

The next step is to loop through the `faceArr` array and compare the name of each learned face to the value of `you`. If a match is found, we update the value of the global `onList` variable to `true`. Create a `for` loop to check each item in `faceArr` against `you`. Inside this loop, use an `if` statement to update the value of the `onList` variable to `true` if a match is found.

```
async function openCallback() {
    socket.Unsubscribe("FaceRecognition");
    await sleep(3000);

    axios.get("http://" + ip + "/api/faces").then(function (res) {
        let faceArr = res.data.result;
        console.log("Learned faces:", faceArr);

        // Loop through each item in faceArr.
        // Compare each item to the value stored
        // in the you variable.
            for (let i = 0; i < faceArr.length; i++) {
            // If a match is found, update
            // the value of onList to true.
            if (faceArr[i] === you) {
                onList = true;
            }
        }
    });
}
```

JavaScript

At this point the program takes one of two paths. If `onList` becomes `true`, Misty should start facial recognition to identify the user in her field of vision and greet them by name. Otherwise, Misty should start facial training, so she can learn the user's face and recognize them in the future. Set aside a section of the script for `/* COMMANDS */` and declare two new functions, `startFaceRecognition()` and `startFaceTraining()`, for each of these paths. Use the prefix `async` when you declare the `startFaceTraining()` function to indicate the function is asynchronous.

```
/* COMMANDS */

// Define the function that executes
// if the value stored in you is on
// Misty's list of known faces.
function startFaceRecognition() {

};

// Define the function that executes
// to learn the user's face if the
```

```
// value stored in you is not on Misty's
// list of known faces.
async function startFaceTraining() {

};
```

JavaScript

In either case, we need to subscribe to the `FaceRecognition` WebSocket to receive facial data from Misty. In the `openCallback()` function, after the `for` loop has checked through the list of returned faces, call `socket.Subscribe()`. As described in the second tutorial above, `socket.Subscribe()` accepts eight parameters. Pass `"FaceRecognition"` for the `eventName` and `msgType` parameters. Set `debounceMs` to `200`, and pass a callback function named `_FaceRecognition()` for the `callback` parameter. There is no need to define event conditions for this data stream; pass `null` for all other arguments.

```
async function openCallback() {
    socket.Unsubscribe("FaceRecognition");
    await sleep(3000);

    axios.get("http://" + ip + "/api/faces").then(function (res) {
        let faceArr = res.data.result;
        console.log("Learned faces:", faceArr);

        for (let i = 0; i < faceArr.length; i++) {
            if (faceArr[i] === you) {
                onList = true;
            }
        }

        // Subscribe to the FaceRecognition WebSocket.
        // Pass FaceRecognition for the eventName and
        // msgType parameters. Set debounceMs to 200
        // and pass a callback function named _FaceRecognition
        // for the callback parameter. Pass null for
        // all other arguments.
        socket.Subscribe("FaceRecognition", "FaceRecognition", 200, null,
null, null, null, _FaceRecognition);

    });
}
```

JavaScript

After subscribing to `FaceRecognition`, write an `if...else` statement to execute `startFaceRecognition()` if `onList` is `true`, and to execute `startFaceTraining()` if `onList` is `false`. In each condition, print a message to the console to state whether the program found the user on the list.

```
async function openCallback() {
    socket.Unsubscribe("FaceRecognition");
    await sleep(3000);

    axios.get("http://" + ip + "/api/faces").then(function (res) {
        let faceArr = res.data.result;
        console.log("Learned faces:", faceArr);

        for (let i = 0; i < faceArr.length; i++) {
            if (faceArr[i] === you) {
                onList = true;
            }
        }

        socket.Subscribe("FaceRecognition", "FaceRecognition", 200, null,
null, null, null, _FaceRecognition);

        // Use an if, else statement to execute
        // startFaceRecognition() if onList is true
```

```
            // and to execute startFaceTraining if otherwise.
            if (onList) {
                console.log("You were found on the list!");
                startFaceRecognition();
            } else {
                console.log("You're not on the list...");
                startFaceTraining();
            }

        });
    }
```

JavaScript

COMMANDS

Within the `startFaceRecognition()` function, print a message to the console that Misty is “starting face recognition”. Then, use Axios to send a POST request to the endpoint for the `StartFaceRecognition` command: `"http://" + ip + "/api/faces/recognition/start"`. There is no need to send data along with this request, so you can omit the second parameter of `axios.post()`.

This command tells Misty to start the occipital camera so she can match the face in her field of vision with a name on her list of known faces. Because this is a `FaceRecognition` event, the callback for the `FaceRecognition` WebSocket triggers as this data comes in. If the face is recognized, the name of the recognized person is included in the WebSocket data message. Instructions for handling these messages are included in the **Callbacks** section of this tutorial.

```
function startFaceRecognition() {
    // Print a message to the console that Misty
    // is “starting face recognition”. Then, use
    // Axios to send a POST request to the endpoint
    // for the StartFaceRecognition command.
    console.log("starting face recognition");
    axios.post("http://" + ip + "/api/faces/recognition/start");
};
```

JavaScript

In `startFaceTraining()`, log a message to the console that Misty is “starting face training”. Then use Axios to send a POST request to the endpoint for the `StartFaceTraining` command: `"http://" + ip + "api/faces/training/start"`. This command tells Misty to use her occipital camera to learn the user's face and pair it with a `FaceID` so she can recognize it in the future. Send a data object along with the request that includes the key `FaceId` with the value `you` to attach the name stored in `you` to the learned face.

```
async function startFaceTraining() {
    // Print a message to the console that Misty
    // is “starting face training”. Then use Axios
    // to send a POST request to the endpoint for
    // the StartFaceTraining command.
    console.log("starting face training");
    axios.post("http://" + ip + "/api/faces/training/start", { FaceId:
you });
};
```

JavaScript

To give Misty time to learn the user's face, use the helper function `sleep()` to pause execution of the program. Below the POST command, call `sleep()` and pass in the value `20000` for 20 seconds. This gives Misty plenty of time to finish the facial training process. Prefix `sleep()` with the keyword `await`.

```
async function startFaceTraining() {
    console.log("starting face training");
```

```
    axios.post("http://" + ip + "/api/faces/training/start", { FaceId:
you });
    // Give Misty time to complete the face
    // training process. Call sleep and pass
    // in the value 20000 for 20 seconds.
    await sleep(20000);
    // Print a message to the console that
    // face training is complete.
    console.log("face training complete");

};
```

JavaScript

When Misty is done learning the face, we want her to try to recognize it. Below `sleep()`, log a message to the console that face training is complete. Then, use Axios to send a POST request to the endpoint for the `StartFaceRecognition` command.

```
async function startFaceTraining() {
    console.log("starting face training");
    axios.post("http://" + ip + "/api/faces/training/start", { FaceId:
you });
    await sleep(20000);
    console.log("face training complete");
    // Use Axios to send a POST request to the endpoint
    // for the StartFaceRecognition command.
    axios.post("http://" + ip + "/api/faces/recognition/start");
};
```

JavaScript

CALLBACKS

Data sent through the `FaceRecognition` event subscription is passed to the `_FaceRecognition()` callback function. As discussed in previous tutorials, WebSocket connections sometimes send registration and error messages that do not contain event data. To handle messages unrelated to `FaceRecognition` events, wrap the code for the `_FaceRecognition()` callback inside `try` and `catch` statements. As seen in the example, you can print caught errors to the console by passing `e` to the `catch` statement, but this is not necessary for the program to execute successfully.

```
// Define the callback function for handling
// FaceRecognition event data.
function _FaceRecognition(data) {
    // Wrap the code for the _FaceRecognition callback
    // inside try and catch statements.
    try {

    }
    // Print any errors to the console.
    catch (e) {
        console.log("Error: " + e);
    }
}
```

JavaScript

The `_FaceRecognition()` callback triggers any time the occipital camera gathers relevant data. Messages come in regardless of whether Misty recognizes a face she detects. The message returned by the `FaceRecognition` WebSocket includes a `"label"` property. If a detected face cannot be recognized, the value of `"label"` is `"unknown person"`. If a message does not hold any face data, then `"label"` doesn't exist or is `undefined`. In the `_FaceRecognition()` callback function, use an `if` statement to check that `"label"` does not equal any of these values.

```
function _FaceRecognition(data) {
    try {
        // Use an if statement to check that label
        // does not equal "unknown person", null, or
        // undefined. label is included in the
        // message returned by FaceRecognition WebSocket events.
        if (data.message.label !== "unknown person" &&
data.message.label !== null && data.message.label !== undefined) {

        }
    }
    catch (e) {
        console.log("Error: " + e);
    }
}
```

JavaScript

> **Note:** This program does not handle the case where the value of `you` is on the list of known faces, but does not match the face of the person in Misty's field of vision. This tutorial is designed to introduce the basics of face commands and `FaceRecognition` events, and does not address how to handle issues such as the above. This kind of edge case could be handled in a number of ways. For example, you could have Misty print a message that the face does not match the value stored in `you`, and then command her to learn the new face and assign it a numeric value for `FaceID`. Alternately, you could have Misty start face training and include a form in your .html document to allow the user to pass a new value for `FaceID`.

If a face is recognized, the value of the `"label"` property is the name of the recognized person. In our case, this should also be the string stored in `you`. Inside the `if` statement, write code to print a message to greet the recognized face, unsubscribe from `"FaceRecognition"`, and issue a POST request to the endpoint for the command to `StopFacialRecognition`: `"http://" + ip + "/api/faces/recognition/stop"`.

```
function _FaceRecognition(data) {
    try {
        if (data.message.label !== "unknown person" &&
data.message.label !== null && data.message.label !== undefined) {
            // If the face is recognized, print a
            // message to greet the person by name.
            console.log(`A face was recognized. Hello there
${data.message.label}!`);

            // Unsubscribe from the FaceRecognition WebSocket.
            socket.Unsubscribe("FaceRecognition");
            // Use Axios to issue a POST command to the
            // endpoint for the StopFaceRecognition command.
            axios.post("http://" + ip + "/api/faces/recognition/stop");
        }
    }
    catch (e) {
        console.log("Error: " + e);
    }
}
```

JavaScript

At the bottom of the script, call `socket.Connect()`. When the connection is established, the `openCallback()` function executes and the process begins.

```
// Open the connection to your robot.
// When the connection is established,
```

```
// the openCallback function executes.
socket.Connect();
```

JavaScript

When you load the `.html` file in your browser, the program:

- connects with Misty
- sends a `GetKnownFaces` command and checks whether your name is on the list of faces Misty already knows
- subscribes to the `FaceRecognition` WebSocket to receive messages when Misty is commanded to `StartFaceRecognition`
- recognizes and greets you if you are on the list of known faces, or sends a `StartFaceTraining` command to learn your face if you are not

Full Sample

Download the full .html document from GitHub.

Taking Pictures

This tutorial describes how to write a remote-running program for Misty that takes a photo with her 4K camera and saves it to her local storage when she detects a face in her field of vision. It teaches

- how to subscribe to the `FaceRecognition` WebSocket
- how to engage with Misty's face detection capabilities
- how to use the `TakePicture` command to take photos with Misty's 4K camera and save them to your robot
- how to control the flow of a program to trigger commands when specific environmental circumstances are met

Setting Up Your Project

This project uses the Axios library and the `lightSocket.js` helper tool to handle requests and simplify the process of subscribing to Misty's WebSocket connections. You can download this tool from our GitHub repository. Save the `lightSocket.js` file to a “tools” or “assets” folder in your project.

To set up your project, create a new HTML document. Give it a title and include references to `lightSocket.js` and a CDN for the Axios library in the `<head>` section. We write the code for commanding Misty within `<script>` tags in the `<body>` section of this document.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Tutorial | Taking Pictures</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <!-- Include references to a CDN for the Axios library and the local
path where lightSocket.js is saved in the <head> of your document -->
    <script src="https://unpkg.com/axios/dist/axios.min.js"></script>
    <script src="<local-path-to-lightSocket.js>"></script>
</head>
<body>
    <script>
    // The code for commanding Misty goes here!
    </script>
</body>
```

HTML

Writing the Code

Within `<script>` tags in the `<body>` of your document, declare a constant variable `ip` and set its value to a string with your robot's IP address. We use this variable to send commands to Misty. Other global variables are declared later in the project.

```
/* GLOBAL */

// Declare a constant variable.
// Set its value to your robot's IP address.
const ip = <robotipaddress>;
```

JavaScript

Next, define a function called `sleep()`. We use this function to help control the flow of the program. The `sleep()` function creates and returns a promise that resolves when `setTimeout()` expires after a designated number of milliseconds. Define `sleep()` beneath the global variables so it can be referenced throughout the program.

```
/*TIMEOUT */

// Define sleep(). This function pauses execution
// for a set period of time.
function sleep(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}
```

Js

Create an new instance of `LightSocket` called `socket`. This instance of `LightSocket` takes as parameters the IP address of your robot and two optional callback functions (the first triggers when a connection is opened, and the second triggers when it's closed). Pass the `ip` variable and a function called `openCallback()` to `socket` as the first and second parameters.

```
/* GLOBALS */

const ip = "<robotipaddress>";
// Create a new instance of LightSocket.
// Pass the ip variable and the name of the
// callback function to run when the socket opens.
let socket = new LightSocket(ip, openCallback);
```

JavaScript

Declare the `openCallback()` function. Prefix the definition of `openCallback()` with the keyword `async()` to declare it as an asynchronous function and enable the use of the `sleep()` function.

```
/* CALLBACKS */

// Define the openCallback() function.
// This function runs when the socket opens.
async function openCallback() {

}
```

JavaScript

To keep track of whether we are currently subscribed to a `FaceRecognition` event, declare a global variable called `subscribed` near the global `ip` variable.

```
/* GLOBALS */

const ip = "<robotipaddress>";
let socket = new LightSocket(ip, openCallback);
// Track whether we are currently subscribed
```

```
// to FaceRecognition events.
let subscribed;
```

JavaScript

Set the value of `subscribed` to `false` in the beginning of the `openCallback()` function.

```
/* CALLBACKS */

async function openCallback() {
    subscribed = false;
}
```

JavaScript

Each time a picture is taken, we unsubscribe from the `FaceRecognition` WebSocket, pause execution, and re-subscribe to the WebSocket. We do this to prevent Misty from taking dozens of pictures of the same person every time she detects a face. To manage this, we send a command to unsubscribe from the `"FaceRecognition"` event before each attempt to establish a connection.

Inside `openCallback()`, call `socket.Unsubscribe()` and pass in the `"FaceRecognition"` event name. After unsubscribing, call `sleep()` (prefixed with the keyword `await`) and pass in the value `8000`. This tells the program to pause for 8 seconds, which is how long we want Misty to wait before re-subscribing to `FaceRecognition` and sending more face detection event data.

```
/* CALLBACKS */

async function openCallback() {
    subscribed = false;
    // Unsubscribe from the FaceRecognition event.
    socket.Unsubscribe("FaceRecognition");
    // Pause execution while the event subscription ends.
    await sleep(8000);
}
```

JavaScript

Next, call `socket.Subscribe()`. The `socket.Subscribe()` method takes eight arguments. For more information about what each of these arguments does, see the documentation on using the `lightSocket.js` tool here.

```
socket.Subscribe(eventName, msgType, debounceMs, property,
inequality, value, [returnProperty], [eventCallback])
```

JavaScript

When you call `socket.Subscribe()`, pass `"FaceRecognition"` for the `eventName` argument, pass `"FaceRecognition"` for `msgType`, pass `1000` for `debounceMS`, and pass `"_FaceRecognition"` for `eventCallback`. Pass `null` for all other arguments.

```
/* CALLBACKS */

async function openCallback() {
    subscribed = false;
    socket.Unsubscribe("FaceRecognition");
    await sleep(8000);
    // Call socket.Subscribe(). Pass in the following
    // arguments to subscribe to "FaceRecognition" events.
    socket.Subscribe("FaceRecognition", "FaceRecognition", 1000, null,
null, null, null, _FaceRecognition);

}
```

JavaScript

Use the keyword `async` to define the `_FaceRecognition()` callback that runs when a `FaceRecognition` event triggers. This function takes a `data` argument, which holds the data from the event message. Write code to print a message to the console each time the callback triggers, including the message response data.

```
// Define the _FaceRecognition() callback function.
// This function handles FaceRecognition event data.
async function _FaceRecognition(data) {
    // Print a message each time the callback executes.
    console.log("CV callback called: ", data);
```

JavaScript

When we establish a connection, we want to update the value of `subscribed` to reflect that we are subscribed to the event. Use an `if` statement to check if `subscribed` is `false`. If it is, set it to `true`.

```
async function _FaceRecognition(data) {
    console.log("CV callback called: ", data);
    // Update subscribed to true
    if (!subscribed) {
        subscribed = true;
    }
}
```

JavaScript

As Misty takes pictures of the faces she recognizes, we unsubscribe and re-subscribe to "FaceRecognition". However, because it's okay for face detection to remain active even when we are not subscribed to `"FaceRecognition"` event messages, we only need to send the command to start face detection once. We can accomplish this by using a global variable called `firstTime` that we initialize with a value of `true`.

```
/* GLOBALS */
const ip = "<robotipaddress>";
// Set firstTime to true. This variable tracks
// whether we have already sent the command to
// start face detection.
let firstTime = true;
let subscribed;
let socket = new LightSocket(ip, openCallback);
```

JavaScript

When the callback triggers, use an `if` statement to check if `firstTime` is `true`. If it is, send a POST request to the endpoint for the `StartFaceDetection` command. Use `catch()` to handle and log any errors you receive when sending the command. Set `firstTime` to `false` and leave it that way for the remainder of the program's execution.

```
async function _FaceRecognition(data) {
    console.log("CV callback called: ", data);
    if (!subscribed) {
        subscribed = true;
        // If firstTime is true, send a POST request
        // to the endpoint for the StartFaceDetection
        // command.
        if (firstTime) {
            axios.post("http://" + ip + "/api/faces/recognition/start")
                .catch((err) => {
                    console.log(err);
                });
            // Update firstTime
            firstTime = false;
        }
```

```
        }
    }
```

JavaScript

The first message we receive when we subscribe to the `FaceRecognition` WebSocket is a registration message that does not contain data relevant to our program. When the `_FaceRecognition()` callback triggers for the first time, we want to send the command to start face detection, but we want to prevent execution of the rest of the code to avoid processing this registration message. To do this, within the `if` statement checking the value of `subscribed`, use `return` to exit the callback and take no further action.

```
async function _FaceRecognition(data) {
    console.log("CV callback called: ", data);
    if (!subscribed) {
        subscribed = true;
        if (firstTime) {
            axios.post("http://" + ip + "/api/faces/recognition/start")
                .catch((err) => {
                    console.log(err);
                });
            firstTime = false;
            }
            // Exit the callback.
            return
        }
    }
```

JavaScript

The rest of the callback function handles cases where relevant data comes through. This occurs whenever Misty detects a face in her field of vision. Because the program pauses each time a picture is taken, this section of the callback doesn't execute more frequently than every 8 seconds.

To have Misty take a picture, use `axios.get()` to send a GET request to the endpoint for the `TakePicture` command. This endpoint accepts values for parameters that specify whether the image data should be returned as a Base64 string, what name the image file should be given, what size the image should be, whether to display the image on Misty's screen, and whether to overwrite an image with the same file name if one exists on your robot. Read the documentation on this endpoint for detailed descriptions of these parameters. When you call `axios.get()`, pass in the endpoint for the `TakePicture` command as the first argument. For the second argument, pass in a `params` object with the following key, value pairs:

- Set `Base64` to `null`. This tells Misty not to return the image data as a base64 string.
- Set `FileName` to the variable `fileName`. Declaring a value for this parameter tells Misty to save the photo to her file system. The photo is saved with a name that matches the value stored in the `fileName` variable, which is defined later in this project.
- Set `Width` and `Height` to `1200` and `1600`, respectively. These sizes match the resolution of the photo taken by the 4K camera.
- Set `DisplayOnScreen` to `false`. We don't want Misty to display these photos on her screen after she takes them.
- Set `OverwriteExisting` to `true` so Misty overwrites any old images that have the same name as newly captured photos.

```
async function _FaceRecognition(data) {
    console.log("CV callback called: ", data);
    if (!subscribed) {
        subscribed = true;
        if (firstTime) {
            axios.post("http://" + ip + "/api/faces/recognition/start")
                .catch((err) => {
                    console.log(err);
                });
            firstTime = false;
```

```
            }
            return
        }
        // Use axios.get() to send a GET request
        // to the endpoint for the TakePicture command.
        // Pass in the following params to save the file
        // to Misty.
        axios.get("http://" + ip + "/api/cameras/rgb", {
            params: {
                Base64: null,
                FileName: fileName,
                Width: 1200,
                Height: 1600,
                DisplayOnScreen: false,
                OverwriteExisting: true
            }
        })
 }
```

JavaScript

Use a `then()` method to log the response, as well as a message indicating the image has been saved with the specified file name.

```
async function _FaceRecognition(data) {
    console.log("CV callback called: ", data);
    if (!subscribed) {
        subscribed = true;
        if (firstTime) {
            axios.post("http://" + ip + "/api/faces/recognition/start")
                .catch((err) => {
                    console.log(err);
                });
            firstTime = false;
        }
        return
    }
    axios.get("http://" + ip + "/api/cameras/rgb", {
        params: {
            Base64: null,
            FileName: fileName,
            Width: 1200,
            Height: 1600,
            DisplayOnScreen: false,
            OverwriteExisting: true
        }
    })
    // Use then() to log the response.
        .then(function (res) {
            console.log(res);
            console.log("Image saved with fileName: '" + fileName + "'");
        });
 }
```

JavaScript

We define `fileName` above this GET request. For this project, we want Misty to take pictures and save them with the date and time the photo was taken. To accomplish this, we use the JavaScript built-in `Date` object. Instantiate a new `Date` object, then call the method `toLocaleString()` to convert the date and time into a string. Windows systems omit certain characters from file names, so we need to use the `replace()` method and pass in some regular expressions to modify the string to an acceptable format and make it easier to read. (**Note:** This code is okay to leave in your program if you are running it on a Mac or Unix system.) These regular expressions replace semicolons with periods, replace spaces with underscores, remove commas, and append the file name with `"_Face"` to indicate that these are images of faces.

```
async function _FaceRecognition(data) {
    console.log("CV callback called: ", data);
    if (!subscribed) {
        subscribed = true;
        if (firstTime) {
            axios.post("http://" + ip + "/api/faces/recognition/start")
                .catch((err) => {
                    console.log(err);
                });
            firstTime = false;
        }
        return
    }
    // Use the Date() object to define a unique name
    // for each picture Misty takes.
    let fileName = new Date().toLocaleString().replace(/[/]/g,
"-").replace(/[:]/g, ".").replace(/[ ]/g, "_").replace(",", "") + "_Face";

    axios.get("http://" + ip + "/api/cameras/rgb", {
        params: {
            Base64: null,
            FileName: fileName,
            Width: 1200,
            Height: 1600,
            DisplayOnScreen: false,
            OverwriteExisting: true
        }
    })
        .then(function (res) {
            console.log(res);
            console.log("Image saved with fileName: '" + fileName + "'");
        });
}
```

JavaScript

After the GET request, call `openCallback()` to start the process over again. To catch and log errors, wrap a `try, catch` statement around the code block that defines the value of `fileName`, makes the GET request, and repeats the call to `openCallback()`.

```
async function _FaceRecognition(data) {
    console.log("CV callback called: ", data);
    if (!subscribed) {
        subscribed = true;
        if (firstTime) {
            axios.post("http://" + ip + "/api/faces/recognition/start")
                .catch((err) => {
                    console.log(err);
                });
            firstTime = false;
        }
        return
    }
    // Wrap the GET request code block in a try, catch statement
    try {
        let fileName = new Date().toLocaleString().replace(/[/]/g,
"-").replace(/[:]/g, ".").replace(/[ ]/g, "_").replace(",", "") + "_Face";
        axios.get("http://" + ip + "/api/cameras/rgb", {
            params: {
                Base64: null,
                FileName: fileName,
                Width: 1200,
                Height: 1600,
                DisplayOnScreen: false,
                OverwriteExisting: true
            }
        })
```

```
                .then(function (res) {
                    console.log(res);
                    console.log("Image saved with fileName: '" + fileName +
"'");
                });
            // Call openCallback to start the process over again
            openCallback();
        }
        catch (err) {
            console.log(err);
        }
}
```

JavaScript

At the end of the program, call `socket.Connect()` to open the connection to the websocket.

```
socket.Connect();
```

JavaScript

Congratulations! You've written a program for Misty to take a photo whenever she detects a face.

- When the document loads, the program establishes a connection to the `FaceRecognition` WebSocket.
- Misty starts face detection and, each time she sees a face, takes a photo with her 4K camera.
- These photos are saved to Misty's local storage and given file names to indicate the date and time when the face was detected and the photo was taken.
- The flow of the program is managed by global variables indicating the status of the WebSocket subscription and whether Misty has already started face recognition.
- Each time a photo is taken, the program unsubscribes from the WebSocket, pauses for a few seconds, and then re-subscribes to the WebSocket connection to start the whole process over again.

Full Sample

Download the full .html document from GitHub.